

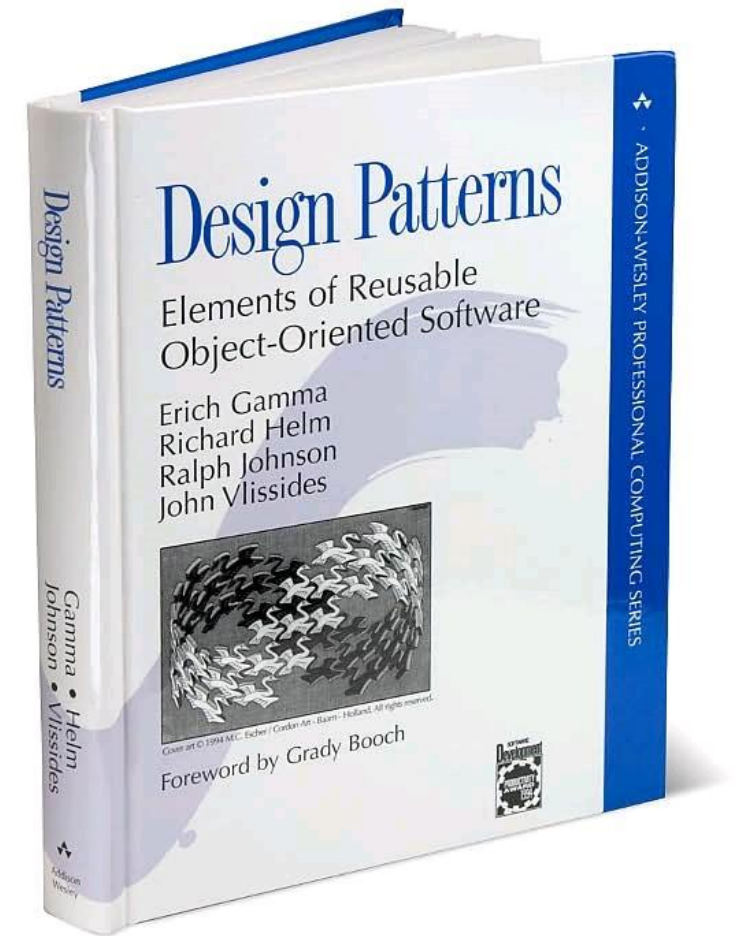
CSC3209 Software architecture and Design Patterns

Lecture13: Design Patterns Behavioural Patterns

Subject Lecturer: Dr. Muhammed Basheer Jasser

References

- Materials from
 - Gamma, Erich. Design patterns: elements of reusable object-oriented software. Pearson Education India, 1995.



Lecture Outline

- What is a Design Pattern
- Design Pattern Catalogue
 - Creational Patterns
 - Structural Patterns
 - **Behavioural Patterns**

Behavioural Patterns

- Behavioral patterns are concerned with algorithms and the assignment of responsibilities between objects.
- Behavioral patterns describe not just patterns of objects or classes but also the patterns of communication between them.
- These patterns characterize complex control flow that's difficult to follow at run-time.

Behavioral Patterns

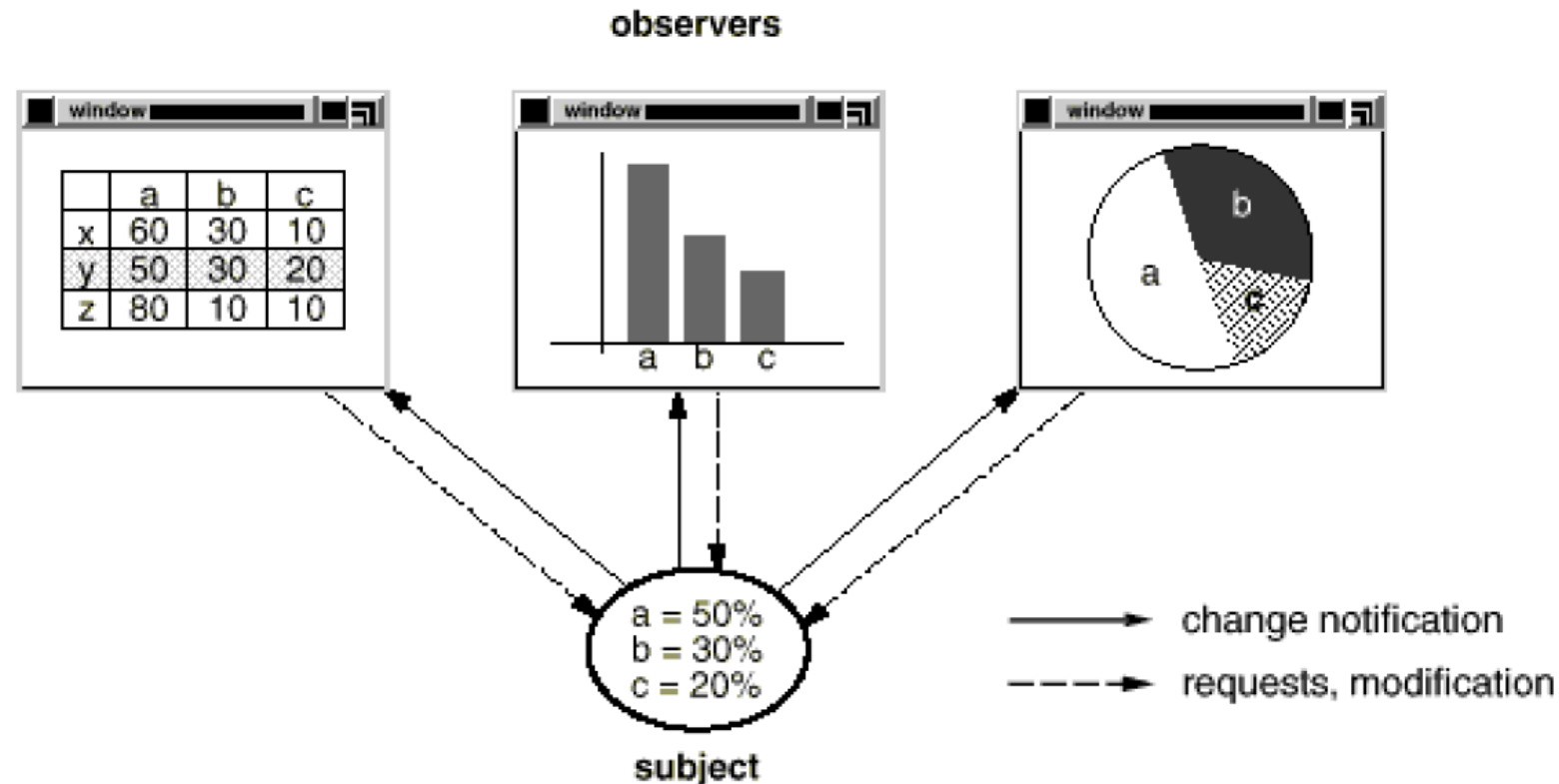
1. Chain of Responsibility
2. Command
3. Interpreter
4. Iterator
5. Mediator
6. Memento
- 7. Observer**
- 8. State**
9. Strategy
10. Template method
11. Visitor

Observer : Behavioural Pattern

- **Intent:** Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
- **Also Known As:** Dependents, Publish-Subscribe
- **Motivation(1):**
 - A common side-effect of partitioning a system into a collection of cooperating classes is the need to maintain consistency between related objects. You don't want to achieve consistency by making the classes tightly coupled, because that reduces their reusability.
 - For example, many graphical user interface toolkits separate the presentational aspects of the user interface from the underlying application data
 - Classes defining application data and presentations can be reused independently. They can work together, too.
 - Both a spreadsheet object and bar chart object can depict information in the same application data object using different presentations. The spreadsheet and the bar chart don't know about each other, thereby letting you reuse only the one you need. But they behave as though they do. When the user changes the information in the spreadsheet, the bar chart reflects the changes immediately, and vice versa.

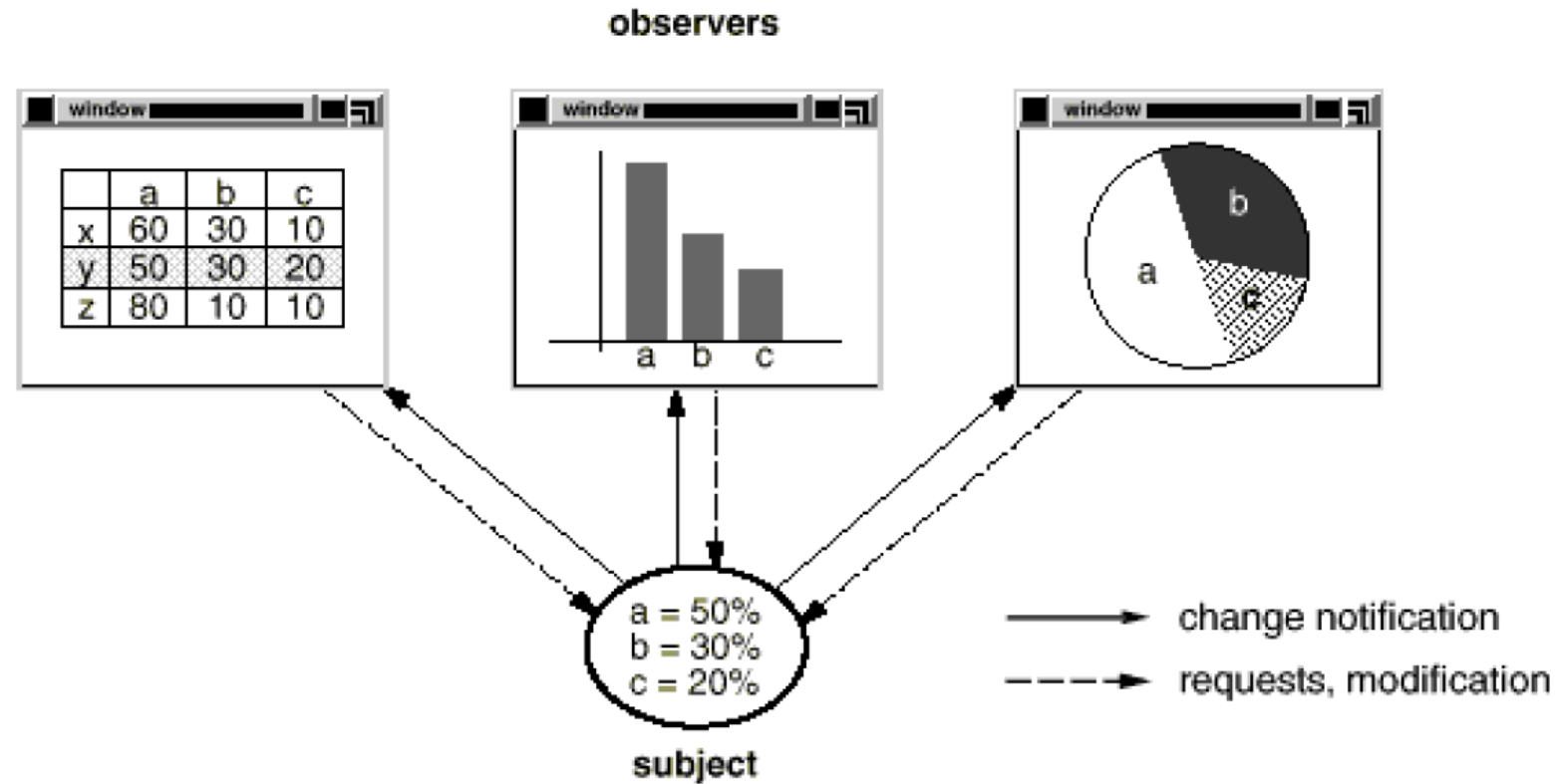
Observer : Behavioural Pattern

- **Motivation(2):**
 - This behavior implies that the **spreadsheet** and **bar chart** are **dependent** on the **data object** and therefore **should be notified** of any **change in its state**.
 - And there's **no reason** to **limit** the **number** of **dependent objects** to two; there **may be any number** of **different user interfaces** to the **same data**.



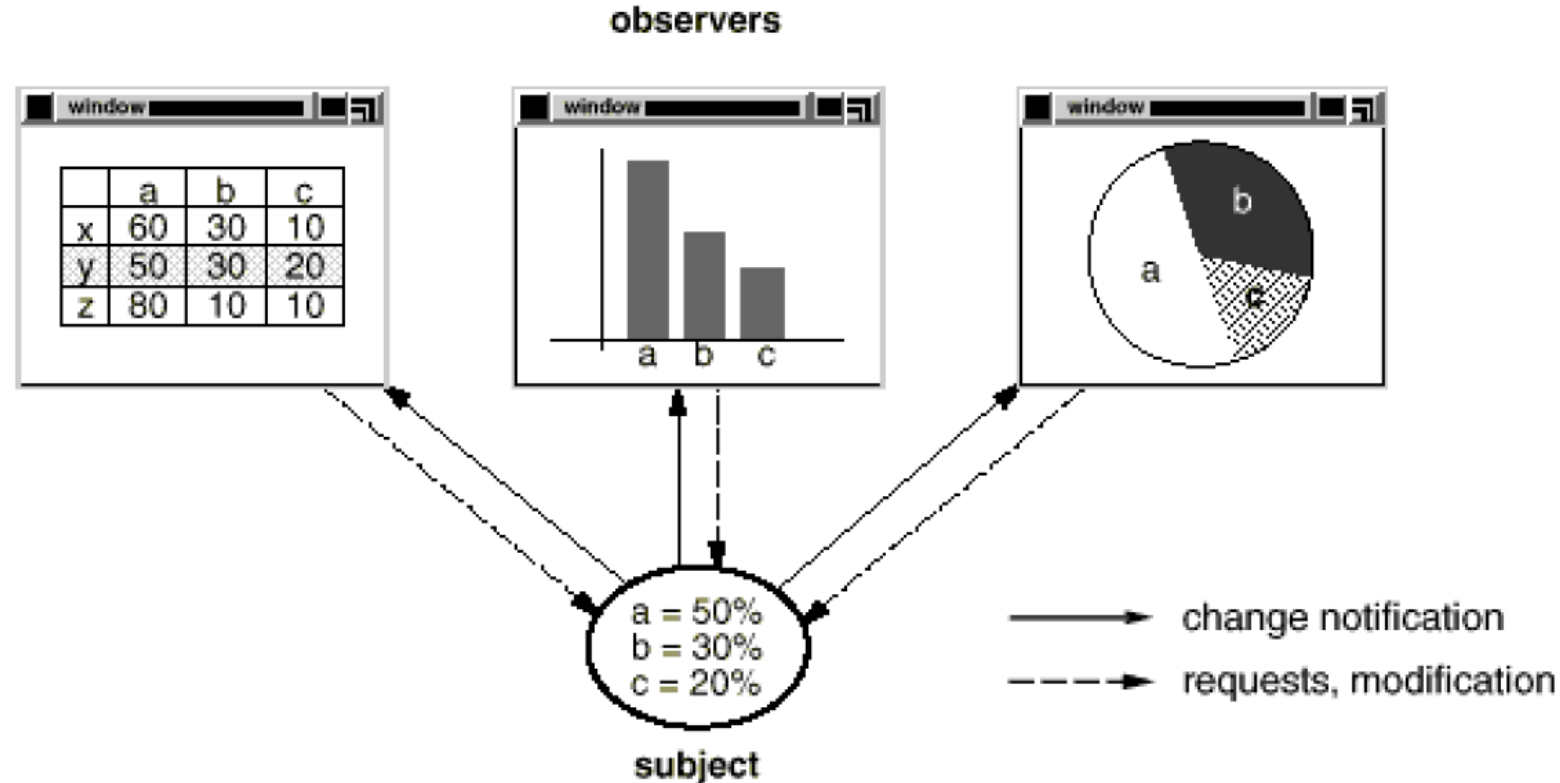
Observer : Behavioural Pattern

- **Motivation(3):**
 - The **Observer pattern describes** how to establish **these relationships**.
 - The **key objects** in this pattern are **subject** and **observer**.
 - A **subject** may have **any number** of **dependent observers**. All **observers** are **notified** **whenever** the **subject undergoes** a **change** in **state**. In **response**, each **observer** will **query** the **subject** to **synchronize** its **state with** the **subject's state**.



Observer : Behavioural Pattern

- **Motivation(4):**
 - This kind of **interaction** is also **known** as **publish-subscribe**. The **subject** is the **publisher** of notifications. It **sends** out these **notifications without** having to **know** who its **observers** are. Any number of **observers** can **subscribe** to **receive notifications**.

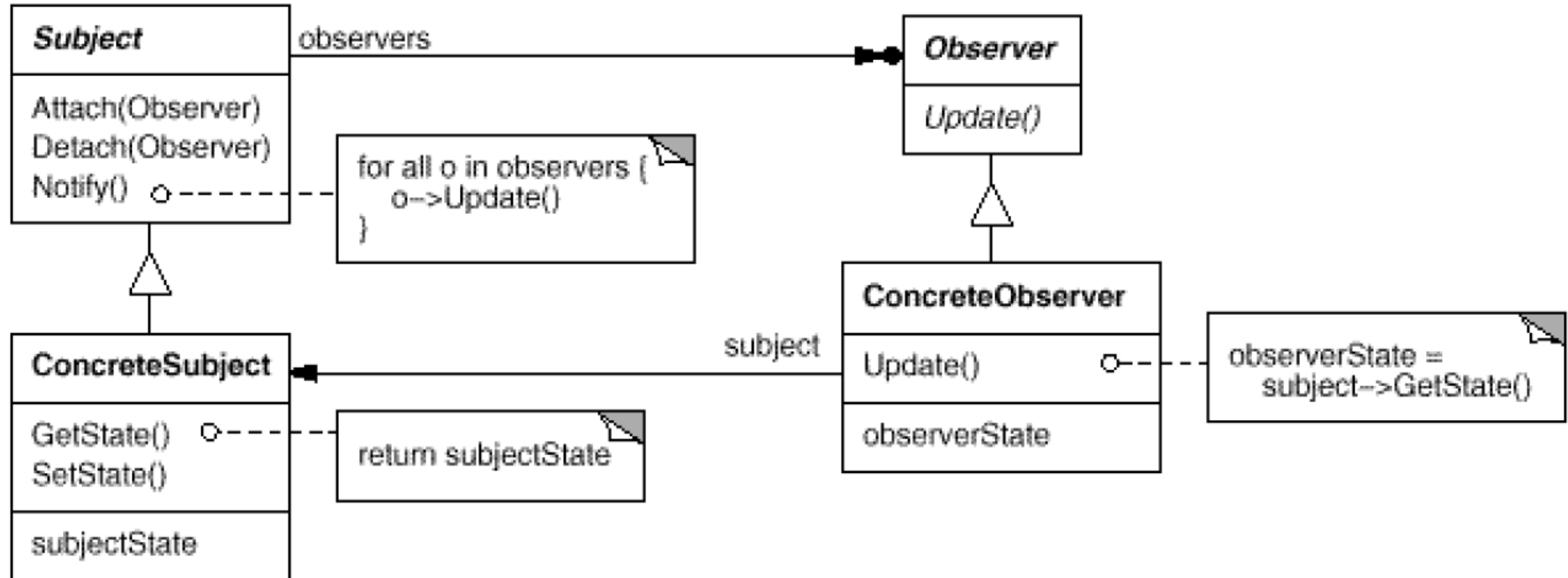


Observer : Behavioural Pattern

- **Applicability:**
 - When an abstraction has two aspects, one dependent on the other. Encapsulating these aspects in separate objects lets you vary and reuse them independently.
 - When a change to one object requires changing others, and you don't know how many objects need to be changed.
 - When an object should be able to notify other objects without making assumptions about who these objects are. In other words, you don't want these objects tightly coupled.

Observer : Behavioural Pattern

- Structure:



Observer : Behavioural Pattern

- **Participants:**
 - **Subject**
 - knows its observers. Any number of Observer objects may observe a subject.
 - provides an interface for attaching and detaching Observer objects.
 - **Observer**
 - defines an updating interface for objects that should be notified of changes in a subject.
 - **ConcreteSubject**
 - stores state of interest to ConcreteObserver objects.
 - sends a notification to its observers when its state changes.
 - **ConcreteObserver**
 - maintains a reference to a ConcreteSubject object.
 - stores state that should stay consistent with the subject's.
 - implements the Observer updating interface to keep its state consistent with the subject's.

Observer : Behavioural Pattern

- **Collaborations:**
 - Concrete Subject notifies its observers whenever a change occurs that could make its observers' state inconsistent with its own.
 - After being informed of a change in the concrete subject, a ConcreteObserver object may query the subject for information. ConcreteObserver uses this information to reconcile its state with that of the subject

Observer : Behavioural Pattern: Example

- We want to implement a real life example of a **Public Profile** of a **celebrity** in **Facebook** who has many **followers**.
- Each of these **followers** wants to **get** all the latest **updates** of his/her favourite **celebrity**. Hence, he/she can follow the celebrity as long as his/her interest persists. When he loses interest, he simply stops following that celebrity.
- Here **followers** are **observers** and **celebrity** is a **subject**.

Observer : Behavioural Pattern: Example

```
package ObserverExample;  
//This Class handles adding, deleting  
//and updating all observers  
public interface Subject {  
    public void register(Observer o);  
    public void unregister(Observer o);  
    public void notifyAllObservers(String s);  
}
```

```
package ObserverExample;  
//The Observers update method is called  
//when the Subject changes  
interface Observer{  
    public void update(String name, String s);  
}
```

Observer : Behavioural Pattern: Example

```
package ObserverExample;
import java.util.ArrayList;
//This class implements Subject interface.
class Celebrity implements Subject {
    private String celebrityName; //name of the celebrity
    private ArrayList<Observer> followers; //list of followers
    public Celebrity(String celebrityName) {
        this.celebrityName = celebrityName;
        followers = new ArrayList<Observer>();
    }
    //add follower to the celebrity's registered follower list
    @Override
    public void register(Observer o) {
        followers.add(o);
        System.out.println(o + " has started following " + celebrityName);
    }
    //remove follower from celebrity's registered follower list
    @Override
    public void unregister(Observer o) {
        followers.remove(o);
        System.out.println(o + " has stopped following " + celebrityName);
    }
    //Notify all the registered followers
    @Override
    public void notifyAllObservers(String post) {
        for(Observer follower : followers)
        {
            follower.update(celebrityName, post);
        }
        System.out.println();
    }
    //This method updates the post.
    //It will internally call notifyAllObservers(post) method
    //after updating the post.
    public void post(String post)
    {
        System.out.println("\n" + celebrityName + " has posted :: " + post + "\n");
        notifyAllObservers(post);
    }
}
```

```
package ObserverExample;
//This class implements Observer interface.
class Follower implements Observer{
    private String followerName;
    public Follower(String followerName) {
        this.followerName = followerName;
    }
    //This method will be called to update all followers
    //regarding the new post posted by celebrity.
    @Override
    public void update(String celebrityName , String post) {
        System.out.println(followerName + " has received " +
            celebrityName + "'s post :: " + post);
    }
    @Override
    public String toString() {
        return followerName;
    }
}
```


Observer : Behavioural Pattern: Example

```
package ObserverExample;
public class ObserverDesignPattern {
public static void main(String[] args) {
    Celebrity mahatir = new Celebrity("Dr.Mahatir Mohammad");
    Celebrity billgates = new Celebrity("Bill Gates");
    Follower muhammad = new Follower("muhammad");
    Follower lee = new Follower("lee");
    Follower tan = new Follower("tan");
    Follower samir = new Follower("samir");

    mahatir.register(muhammad);
    mahatir.register(lee);
    mahatir.register(tan);

    billgates.register(muhammad);
    billgates.register(lee);
    billgates.register(samir);

    mahatir.post("Media Statement: Prime Minister's nomination");
    billgates.post("Here are five books to read this summer if you want to "
        + "learn more about pandemics");
    billgates.unregister(samir);
    billgates.post("This hero is playing a critical role in Kenya's"
        + " preparations to tackle COVID-19 by scaling up medical "
        + "oxygen supplies that will be needed to keep the most "
        + "critically ill patients alive.");
    }
}
```

Observer : Behavioural Pattern: Example

Console

<terminated> ObserverDesignPattern [Java Application] C:\Program Files\Java\jre1.8.0_211\bin\javaw.exe (Jul 7, 2020, 1:06:25 PM)

muhammad has started following Dr.Mahatir Mohammad

lee has started following Dr.Mahatir Mohammad

tan has started following Dr.Mahatir Mohammad

muhammad has started following Bill Gates

lee has started following Bill Gates

samir has started following Bill Gates

Dr.Mahatir Mohammad has posted :: Media Statement: Prime Minister's nomination

muhammad has received Dr.Mahatir Mohammad's post :: Media Statement: Prime Minister's nomination

lee has received Dr.Mahatir Mohammad's post :: Media Statement: Prime Minister's nomination

tan has received Dr.Mahatir Mohammad's post :: Media Statement: Prime Minister's nomination

Bill Gates has posted :: Here are five books to read this summer if you want to learn more about pandemics

muhammad has received Bill Gates's post :: Here are five books to read this summer if you want to learn more about pandemics

lee has received Bill Gates's post :: Here are five books to read this summer if you want to learn more about pandemics

samir has received Bill Gates's post :: Here are five books to read this summer if you want to learn more about pandemics

samir has stopped following Bill Gates

Bill Gates has posted :: This hero is playing a critical role in Kenya's preparations to tackle COVID-19 by scaling up medical

muhammad has received Bill Gates's post :: This hero is playing a critical role in Kenya's preparations to tackle COVID-19 b

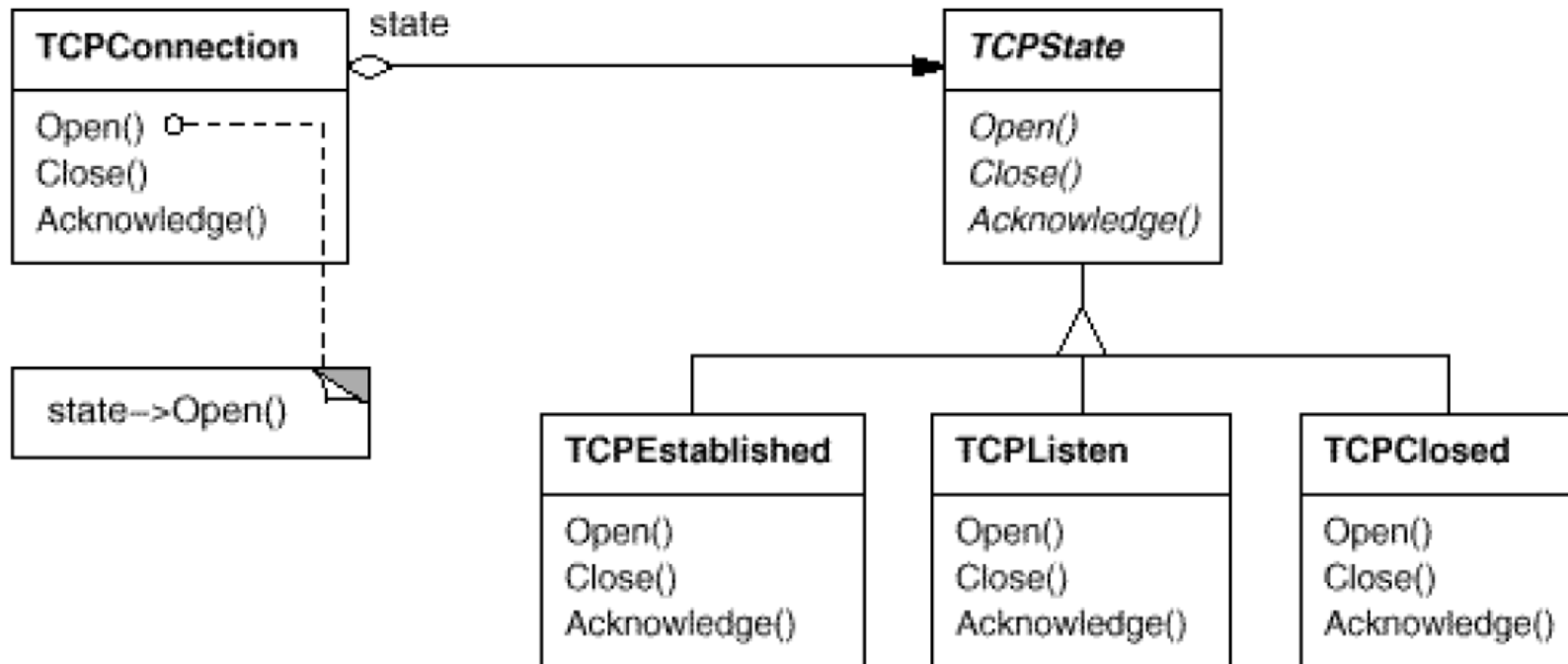
lee has received Bill Gates's post :: This hero is playing a critical role in Kenya's preparations to tackle COVID-19 by sca

State : Behavioural Pattern

- **Intent:** Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.
- **Also Known As:** Objects for States
- **Motivation(1):**
 - Consider a class `TCPConnection` that represents a network connection.
 - A `TCPConnection` object can be in one of several different states: `Established`, `Listening`, `Closed`.
 - When a `TCPConnection` object receives requests from other objects, it responds differently depending on its current state.
 - For example, the effect of an `Open` request depends on whether the connection is in its `Closed` state or its `Established` state.
 - The `State` pattern describes how `TCPConnection` can exhibit different behavior in each state.

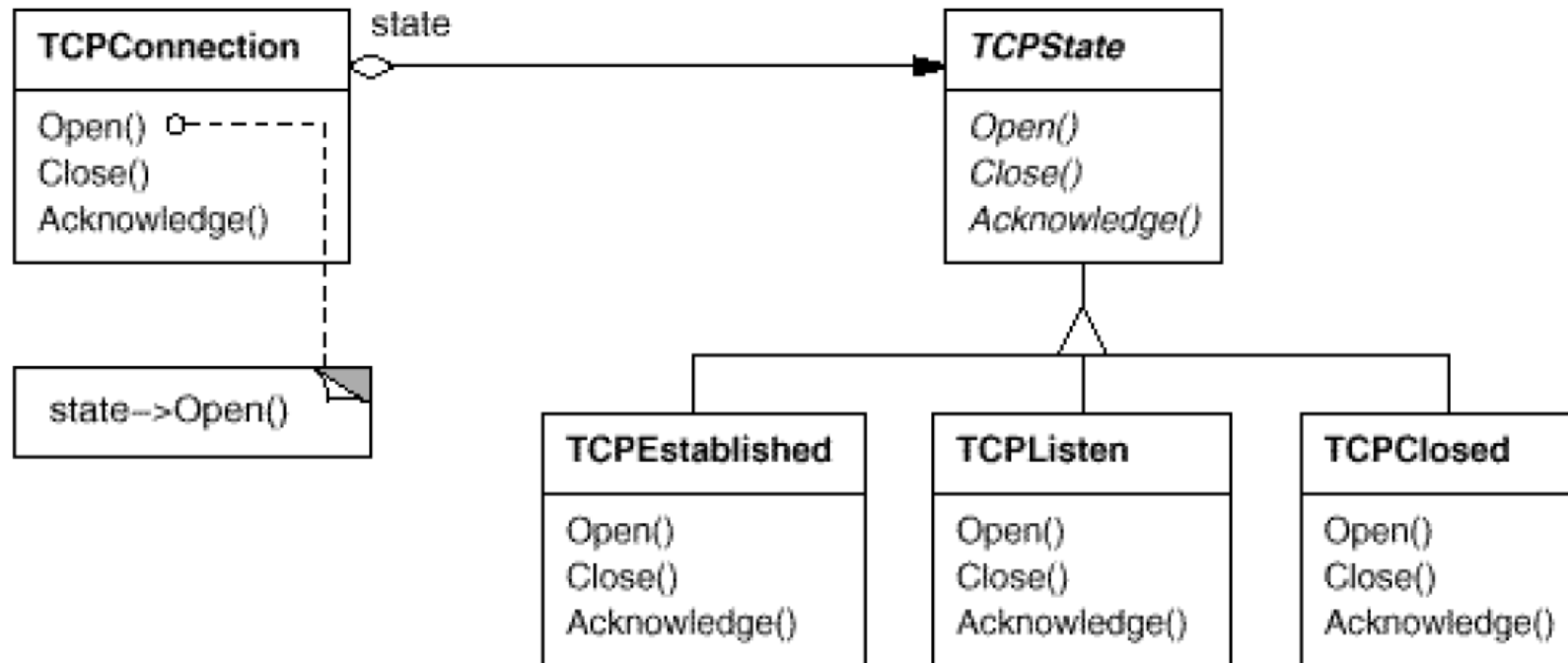
State : Behavioural Pattern

- **Motivation(2):**
 - The **key idea** in this pattern is to **introduce** an **abstract class** called **TCPState** to **represent** the **states** of the network connection.
 - The **TCPState** class **declares** an **interface common** to **all classes** that represent different operational states. **Subclasses** of **TCPState** implement state-specific behavior.
 - For example, the **classes** **TCPEstablished** and **TCPClosed** implement behavior particular to **the Established** and **Closed** states of **TCPConnection**.



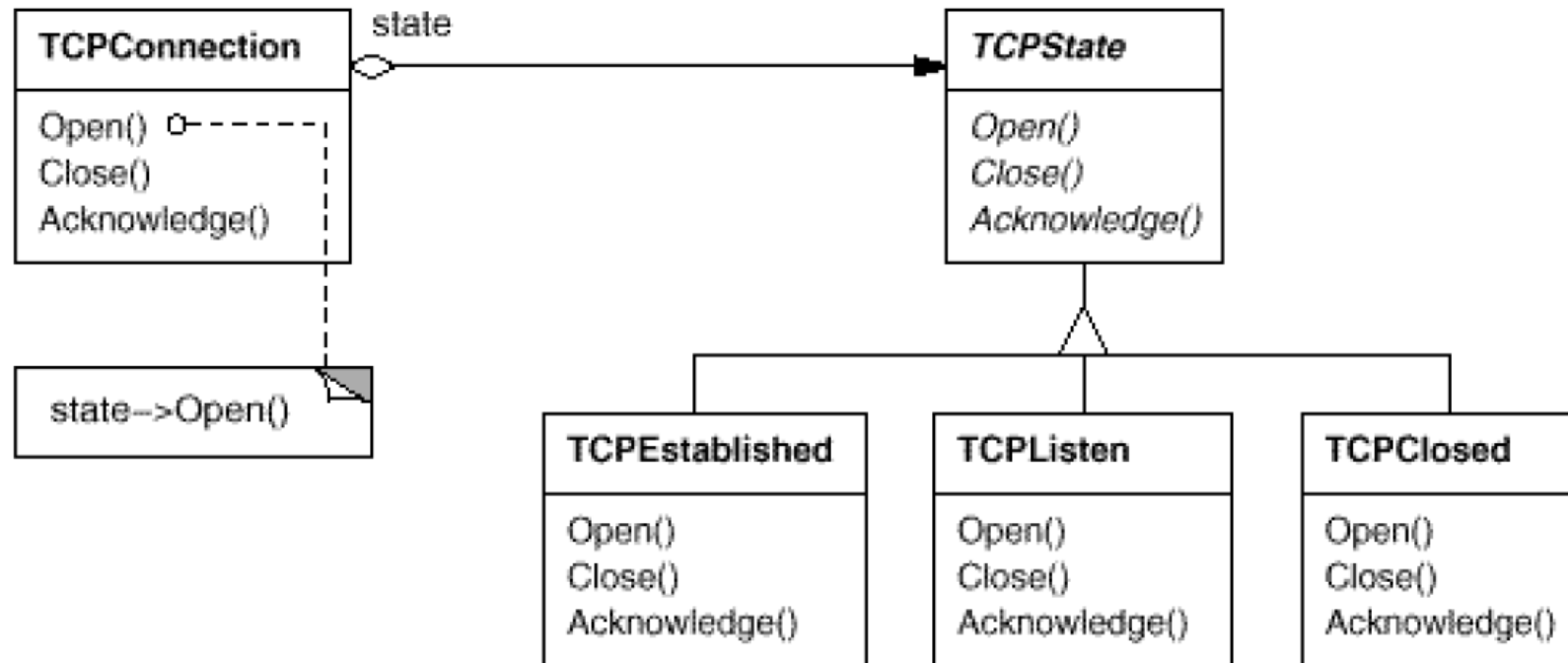
State : Behavioural Pattern

- **Motivation(3):**
 - The class `TCPConnection` maintains a state object (an instance of a subclass of `TCPState`) that represents the current state of the `TCPConnection`.
 - The class `TCPConnection` delegates all state-specific requests to this state object.
 - `TCPConnection` uses its `TCPState` subclass instance to perform operations particular to the state of the connection.



State : Behavioural Pattern

- **Motivation(4):**
 - Whenever the connection changes state, the `TCPConnection` object changes the state object it uses.
 - When the connection goes from established to closed, for example, `TCPConnection` will replace its `TCPEstablished` instance with a `TCPClosed` instance.

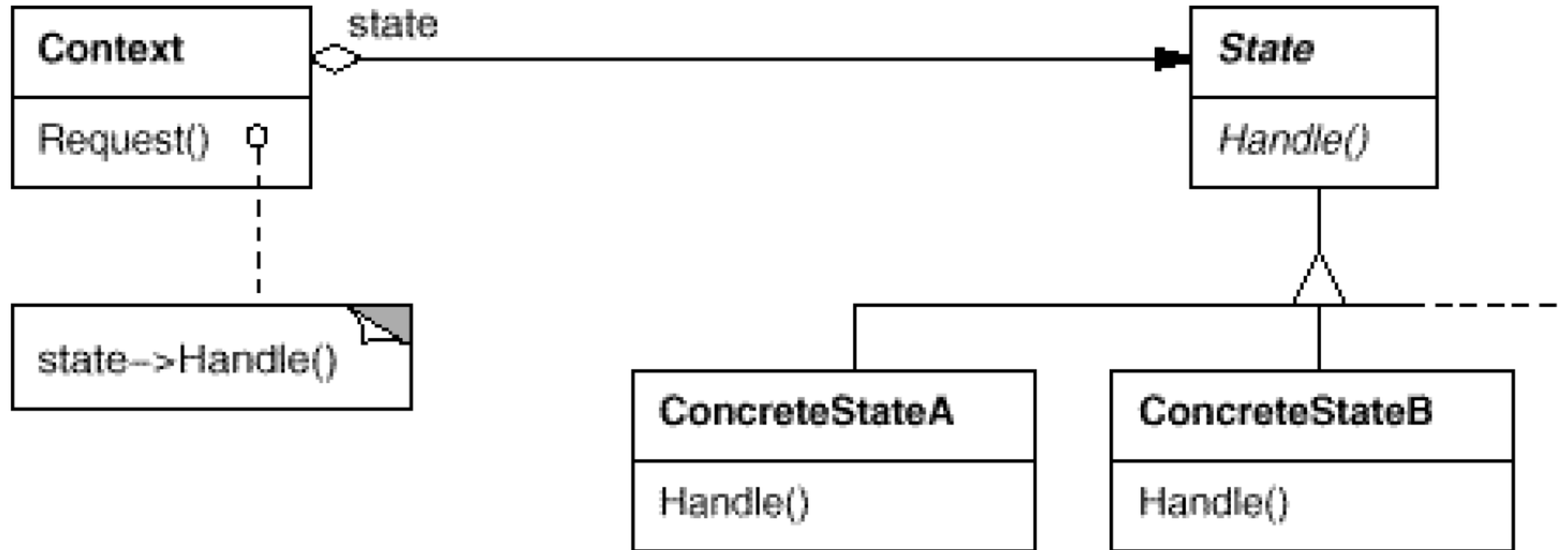


State : Behavioural Pattern

- **Applicability:** Use the State pattern in either of the following cases:
 - An object's behavior depends on its state, and it must change its behavior at run-time depending on that state.
 - Operations have large, multipart conditional statements that depend on the object's state.
 - This state is usually represented by one or more enumerated constants.
 - Often, several operations will contain this same conditional structure. The State pattern puts each branch of the conditional in a separate class. This lets you treat the object's state as an object in its own right that can vary independently from other objects.

State : Behavioural Pattern

- Structure:



State : Behavioural Pattern

- **Participants:**
- **Context (TCPConnection)**
 - defines the interface of interest to clients.
 - maintains an instance of a ConcreteState subclass that defines the current state.
- **State (TCPState)**
 - defines an interface for encapsulating the behavior associated with a particular state of the Context.
- **ConcreteState subclasses** (TCPEstablished, TCPListen, TCPClosed)
 - each subclass implements a behavior associated with a state of the Context.

State : Behavioural Pattern

- **Collaborations:**
- Context delegates state-specific requests to the current ConcreteStateobject.
- A context may pass itself as an argument to the State object handling the request. This lets the State object access the context if necessary.
- Context is the primary interface for clients. Clients can configure a context with State objects. Once a context is configured, its clients don't have to deal with the State objects directly.
- Either Context or the ConcreteState subclasses can decide which state succeeds another and under what circumstances.

State : Behavioural Pattern: Example

- A company is looking to build a robot for cooking. The company wants a simple robot that can simply walk and cook.
- A user can operate a robot using a set of commands via remote control. Currently, a robot can do three things, it can walk, cook, or can be switched off.
- The company has set protocols to define the functionality of the robot.
 - If a robot is in “on” state you can command it to walk. If asked to cook, the state would change to “cook” or if set to “off”, it will be switched off.
 - Similarly, when in “cook” state it can walk or cook, but cannot be switched off.
 - And finally, when in “off” state it will automatically get on and walk when the user commands it to walk but cannot cook in off state.

State : Behavioural Pattern: Example

- This might look like an easy implementation: a robot class with a set of methods like walk, cook, off, and states like on, cook, and off. We can use if-else branches or switch to implement the protocols set by the company. But too much if-else or switch statements will create a maintenance nightmare as complexity might increase in the future.
- You might think that we can implement this without issues using if-else statements, but as a change comes the code would become more complex.
- The requirement clearly shows that the behavior of an object is truly based on the state of that object. We can use the State Design Pattern which encapsulates the states of the object into another individual class and keeps the context class independent of any state change.

State : Behavioural Pattern: Example

```
package StateExample;
public interface RoboticState {
    public void walk();
    public void cook();
    public void off();
}
```

```
package StateExample;
public class RoboticOn implements RoboticState{
    private final Robot robot;
    public RoboticOn(Robot robot){
        this.robot = robot;
    }
    @Override
    public void walk() {
        System.out.println("Walking...");
    }
    @Override
    public void cook() {
        System.out.println("Cooking...");
        robot.setRoboticState(robot.getRoboticCook());
    }
    @Override
    public void off() {
        robot.setRoboticState(robot.getRoboticOff());
        System.out.println("Robot is switched off");
    }
}
```

```
package StateExample;
public class RoboticCook implements RoboticState{
    private final Robot robot;
    public RoboticCook(Robot robot){
        this.robot = robot;
    }
    @Override
    public void walk() {
        System.out.println("Walking...");
        robot.setRoboticState(robot.getRoboticOn());
    }
    @Override
    public void cook() {
        System.out.println("Cooking...");
    }
    @Override
    public void off() {
        System.out.println("Cannot switched off "
            + "while cooking...");
    }
}
```

```
package StateExample;
public class RoboticOff implements RoboticState{
    private final Robot robot;
    public RoboticOff(Robot robot){
        this.robot = robot;
    }
    @Override
    public void walk() {
        System.out.println("Walking...");
        robot.setRoboticState(robot.getRoboticOn());
    }
    @Override
    public void cook() {
        System.out.println("Cannot cook at Off state.");
    }
    @Override
    public void off() {
        System.out.println("Already switched off...");
    }
}
```

State : Behavioural Pattern: Example

```
package StateExample;
public class Robot implements RoboticState{
    private RoboticState roboticOn;
    private RoboticState roboticCook;
    private RoboticState roboticOff;
    private RoboticState state;
    public Robot(){
        this.roboticOn = new RoboticOn(this);
        this.roboticCook = new RoboticCook(this);
        this.roboticOff = new RoboticOff(this);
        this.state = roboticOn;
    }

    public void setRoboticState(RoboticState state){
        this.state = state;
    }
    public RoboticState getState() {
        return state;
    }
    @Override
    public void walk() {
        state.walk();
    }
    @Override
    public void cook() {
        state.cook();
    }
    @Override
    public void off() {
        state.off();
    }
}
```

```
public RoboticState getRoboticOn() {
    return roboticOn;
}
public void setRoboticOn(RoboticState roboticOn) {
    this.roboticOn = roboticOn;
}

public RoboticState getRoboticCook() {
    return roboticCook;
}
public void setRoboticCook(RoboticState roboticCook) {
    this.roboticCook = roboticCook;
}

public RoboticState getRoboticOff() {
    return roboticOff;
}
public void setRoboticOff(RoboticState roboticOff) {
    this.roboticOff = roboticOff;
}
}
```

State : Behavioural Pattern: Example

```
package StateExample;
public class TestStatePattern {
    public static void main(String[] args) {
        Robot robot = new Robot();
        robot.walk();
        robot.cook();
        robot.walk();
        robot.off();
        robot.walk();
        robot.off();
        robot.cook();
    }
}
```

 Console 

<terminated> TestStatePattern [Java Application]

Walking...

Cooking...

Walking...

Robot is switched off

Walking...

Robot is switched off

Cannot cook at Off state.